

API Shape: Memorize Returns		
call	return / meaning	trap
fork()	child sees 0; parent sees child PID; -1 error	both continue after call
execvp(file,argv)	returns only on failure	argv must end with NULL
waitpid(pid,&st,0)	wait/reap one child	pid=-1 means any child
pipe(p)	p[0] read, p[1] write	EOF only when all write ends closed
dup2(old,new)	makes new refer to old target	direction matters

Fully Annotated Shell Command

```
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char *argv[] = {"wc", "-l", "in.txt", NULL};
    pid_t pid = fork();

    if (pid < 0) { perror("fork"); exit(1); }

    if (pid == 0) {
        // Child only: replace child with wc.
        execvp(argv[0], argv);
        // Only reached if execvp failed.
        perror("execvp");
        exit(1);
    }

    // Parent only: wait keeps child from becoming zombie.
    int st;
    waitpid(pid, &st, 0);
    if (WIFEXITED(st)) {
        printf("exit=%d\n", WEXITSTATUS(st));
    }
}
```

- `execvp` keeps same PID but replaces code/heap/stack/globals.
- Parent does not run `wc`; child does.
- `perror/exit` after `execvp` is failure path.

Multiple Fork Trace			
pid_t a = fork();			
pid_t b = fork();			
printf("a&d b=%d\n", a, b);			
process	a	b	why
P0	PID(P1)	PID(P2)	parent of both forks
P1	0	PID(P3)	child of first, parent of second
P2	PID(P1)	0	child of second from P0; copied a
P3	0	0	child of second from P1

Rule: 0 only in the newly-created child of that exact fork.

Conditional Forks: Count Processes			
<pre>if (fork() == 0) { fork(); } printf("x\n");</pre>			
who	first fork	second fork?	prints
original parent	nonzero	no	yes
first child	0	yes, parent side	yes
grandchild	copied 0	created by second	yes

Answer: three prints. Only processes that see first result 0 enter the body.

Child Loop: Correct Pattern

```
for (int i = 0; i < 3; i++) {
    pid_t pid = fork();
    if (pid < 0) exit(1);

    if (pid == 0) {
        do_child_work(i);
        exit(0); // without this, child keeps forking
    }
}

while (waitpid(-1, NULL, 0) > 0) {
    // reap all children
}
```

Exam smell: "create N children." Child must usually stop after its assigned work.

Wait Status: Which Child Failed?

```
for (int i = 0; i < n; i++) {
    pid_t pid = fork();
    if (pid == 0) {
        exit(run_job(i)); // child returns status 0..255
    }
    pids[i] = pid;
}

for (int i = 0; i < n; i++) {
    int st;
    pid_t done = waitpid(pids[i], &st, 0);
    if (done < 0) perror("waitpid");
    if (WIFEXITED(st) && WEXITSTATUS(st) != 0) {
        printf("job %d failed\n", i);
    }
}
```

`waitpid(pids[i],...)` waits for a specific child; `waitpid(-1,...)` waits for whichever finishes next.

Redirect Child Stdout To File

```
int fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
pid_t pid = fork();

if (pid == 0) {
    dup2(fd, STDOUT_FILENO); // fd 1 now points to out.txt
    close(fd); // no longer need original number
    char *args[] = {"echo", "hi", NULL};
    execvp(args[0], args);
    perror("execvp");
    exit(1);
}

close(fd); // parent should close too
waitpid(pid, NULL, 0);

Wrong direction: dup2(STDOUT_FILENO, fd) does not redirect stdout.
```

Use All Three Standard FDs

```
char buf[128];
int n = read(STDIN_FILENO, buf, sizeof(buf));
if (n < 0) {
    write(STDERR_FILENO, "read failed\n", 12);
    exit(1);
}
write(STDOUT_FILENO, buf, n);
```

- `stdin=0`: input source.
- `stdout=1`: normal output.
- `stderr=2`: error output, often not redirected with `stdout`.

Pipeline: Left to Right

```
int p[2];
pipe(p);

pid_t left_pid = fork();
if (left_pid == 0) {
    dup2(p[1], STDOUT_FILENO); // left writes into pipe
    close(p[0]); close(p[1]);
    execvp(left[0], left);
    perror("left"); exit(1);
}

pid_t right_pid = fork();
if (right_pid == 0) {
    dup2(p[0], STDIN_FILENO); // right reads from pipe
    close(p[0]); close(p[1]);
    execvp(right[0], right);
    perror("right"); exit(1);
}

// Parent neither reads nor writes pipeline data.
close(p[0]);
close(p[1]);
waitpid(left_pid, NULL, 0);
waitpid(right_pid, NULL, 0);
```

- If parent keeps `p[1]`, right side may never see EOF.
- Close unused ends after `dup2` in every process.

Pipe EOF Bug: Why It Hangs

```
pipe(p);
if (fork() == 0) {
    close(p[1]);
    while (read(p[0], buf, sizeof(buf)) > 0) {}
    exit(0);
}

// BUG: parent writes but forgets close(p[1])
write(p[1], "abc", 3);
waitpid(-1, NULL, 0); // child waits forever for EOF

Reader sees EOF only after every copy of the write end is closed in every process.
```

Practice Final-Style [Link](#)

```
// incoming -> one -> pipe -> two -> outgoing
static void duet(int incoming, char *one[],
                 char *two[], int outgoing) {
    int p[2]; pid_t pid[2]; pipe(p);

    if ((pid[0] = fork()) == 0) {
        dup2(incoming, STDIN_FILENO);
        dup2(p[1], STDOUT_FILENO);
        close(p[0]); close(p[1]);
        close(incoming); close(outgoing);
        execvp(one[0], one); perror("one"); exit(1);
    }
    close(incoming); close(p[1]);

    if ((pid[1] = fork()) == 0) {
        dup2(p[0], STDIN_FILENO);
        dup2(outgoing, STDOUT_FILENO);
        close(p[0]); close(outgoing);
        execvp(two[0], two); perror("two"); exit(1);
    }
    close(p[0]); close(outgoing);
    waitpid(pid[0], NULL, 0);
    waitpid(pid[1], NULL, 0);
}
```

Hard part: after each `dup2`, close every fd this process does not need. `execvp` preserves the rewired fds.

Command-Line Arguments

```
int main(int argc, char *argv[]) {
    // argv[0] is program name
    // argv[argc] is always NULL
    for (int i = 1; i < argc; i++) {
        printf("arg %d = %s\n", i, argv[i]);
    }
}
```

`argv` is an array of `char *`. Each string is a separate null-terminated char array.

Pipe Without Exec: Direction Sanity

```
pipe(p);
if (fork() == 0) {
    close(p[1]); // child reads only
    char buf[100];
    int n = read(p[0], buf, 99);
    buf[n] = '\0';
    printf("%s\n", buf);
    exit(0);

    close(p[0]); // parent writes only
    write(p[1], "hello", 5);
    close(p[1]); // sends EOF
    waitpid(-1, NULL, 0);
}
```

Use this mental model before adding redirection and `execvp`.

FD Sharing After Fork

```
int fd = open("data", O_RDONLY);
if (fork() == 0) {
    char c; read(fd, &c, 1);
    printf("child %c\n", c);
    exit(0);
}
char c; read(fd, &c, 1);
printf("parent %c\n", c);
waitpid(-1, NULL, 0);
```

- FD table entries are copied.
- They may point to same open-file description.
- Open-file description stores current file offset.
- Scheduling decides who consumes first byte.

Build `argv` For `execvp`

```
char *cmd[] = {
    "grep", // argv[0]: conventionally program name
    "-n",
    "needle",
    "file.txt",
    NULL // required terminator
};
execvp(cmd[0], cmd);

execvp searches PATH for cmd[0] if it has no slash. With ./prog or /bin/ls, it uses that path.
```

Predicted: N-Stage Pipeline

```
int prev_read = input_fd;
for (int i = 0; i < n; i++) {
    int p[2];
    if (i != n-1) pipe(p);
    if (fork() == 0) {
        dup2(prev_read, STDIN_FILENO);
        if (i != n-1) dup2(p[1], STDOUT_FILENO);
        else dup2(output_fd, STDOUT_FILENO);
        close(prev_read);
        if (i != n-1) { close(p[0]); close(p[1]); }
        execvp(cmd[i][0], cmd[i]); exit(1);
    }
    close(prev_read);
    if (i != n-1) { close(p[1]); prev_read = p[0]; }
}
```

Loop invariant: `prev_read` is the stdin source for the next command.

Predicted: Parent Collects Child Output

```
pipe(p);
if (fork() == 0) {
    dup2(p[1], STDOUT_FILENO);
    close(p[0]); close(p[1]);
    execvp(args[0], args); exit(1);
}

close(p[1]);
while ((n = read(p[0], buf, sizeof(buf))) > 0)
    parse_child_output(buf, n);
close(p[0]);
waitpid(-1, NULL, 0);
```

Parent must close its write end before reading to EOF.

Predicted: Wait Any, Map PID To Job

```
for each job i:
    if ((pids[i]=fork()) == 0) { run(i); exit(0); }

int left = n;
while (left > 0) {
    int st; pid_t done = waitpid(-1, &st, 0);
    int i = index_of(pids, done);
    results[i] = WIFEXITED(st) ? WEXITSTATUS(st) : -1;
    left--;
}
```

Use this when completion order is nondeterministic but output array must stay by original job index.

Predicted: Two-Way Parent/Child Pipes

```
int to_child[2], from_child[2];
pipe(to_child); pipe(from_child);
if (fork() == 0) {
    dup2(to_child[0], STDIN_FILENO);
    dup2(from_child[1], STDOUT_FILENO);
    close(to_child[0]); close(to_child[1]);
    close(from_child[0]); close(from_child[1]);
    execvp(cmd[0], cmd); exit(1);
}
close(to_child[0]); close(from_child[1]);
write(to_child[1], request, n);
close(to_child[1]); // child sees EOF on stdin
read(from_child[0], reply, sizeof(reply));
```

Use two pipes for two directions. Never try to use one pipe as bidirectional protocol.

Past Final FD Trap: Close-On-Exec

```
fd1 = open("x", ...);
fd2 = dup(fd1);
set_close_on_exec(fd1); // only fd1 closes on exec
execvp(...);

// Therefore close-on-exec bit is per descriptor-table
// entry, not in shared open-file object.
```

Two descriptors can share offset/file but have different close-on-exec flags.

Monitor API Pattern

```
std::mutex m;
std::condition_variable cv;

void method() {
    std::unique_lock<std::mutex> lock(m);
    while (!predicate_over_shared_state) {
        cv.wait(lock); // unlocks while sleeping; relocks before return
    }
    // update shared state while lock is held
    cv.notify_all(); // or notify_one if one waiter is enough
}
```

CV has no memory. Predicate is the truth; CV only wakes threads to re-check it.

Final Monitor Pattern Map

exam shape	state	wake
CalTrain	waiting, seats, boarding	passengers + train CVs
2022 Merge	waiting per lane, total, current merge	notify one chosen lane
2023 Flight	sorted map position -> waiter CV	notify lowest key
2024 Inventory	counts per stock id	notify all, re-check whole order
Party/Dorm	bucket -> queue of waiter records	match opposite/same bucket
H2O/exact group	queues of waiter records	mark exact records assigned

In every monitor: lock, update waiting state before sleep, while wait, change state before notify, return with invariant true.

Lost Update + Fix

```
int x = 0;
void bad_worker() {
    x++; // load, add, store: not atomic
}

// Bad interleaving:
T1 reads x=0
T2 reads x=0
T1 writes x=1
T2 writes x=1 // one increment lost
```

```
std::mutex m;
void good_worker() {
    std::unique_lock<std::mutex> lock(m);
    x++;
} // auto-unlock
```

Protect the whole read-modify-write, not just the write.

Create Threads And Join

```
void worker(int id) {
    do_work(id);
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 4; i++) {
        threads.push_back(std::thread(worker, i));
    }

    for (std::thread &t : threads) {
        t.join(); // wait for that thread to finish
    }
}
```

join is thread-world waitpid: it waits for completion and cleans up thread resources.

Bounded Buffer

```
class BoundedBuffer {
    std::mutex m;
    std::condition_variable nonempty, nonfull;
    std::queue<int> q;
    int cap;
public:
    void put(int x) {
        std::unique_lock<std::mutex> lock(m);
        while ((int)q.size() == cap) {
            nonfull.wait(lock);
        }
        q.push(x); // state change
        nonempty.notify_one(); // a getter may proceed
    }

    int get() {
        std::unique_lock<std::mutex> lock(m);
        while (q.empty()) {
            nonempty.wait(lock);
        }
        int x = q.front(); // state change
        q.pop();
        nonfull.notify_one(); // a putter may proceed
        return x;
    }
};
```

Two predicates means two CVs are often clearer.

CalTrain Worked Pattern

```
class Station {
    mutex m; condition_variable train, all_boarded;
    int waiting=0, seats=0, boarding=0;
public:
    void load_train(int available) {
        unique_lock<mutex> lock(m);
        seats = available;
        train.notify_all();
        while ((seats > 0 && waiting > 0) || boarding > 0) {
            all_boarded.wait(lock);
        }
        seats = 0;
    }

    void wait_for_train() {
        unique_lock<mutex> lock(m);
        waiting++;
        while (seats == 0) train.wait(lock);
        waiting--; seats--; boarding++;
    } // unlock while passenger physically boards

    void boarded() {
        unique_lock<mutex> lock(m);
        boarding--;
        if ((seats == 0 || waiting == 0) && boarding == 0)
            all_boarded.notify_one();
    }
};
```

Trap: train cannot leave just because seats hit 0; passengers allowed to board still must call boarded().

Bridge / RW Predicate Mini-Patterns

```
// bridge direction fairness
while (crossingOpp > 0 ||
    (waitingOpp > 0 && consecutive_mine >= 5))
    cv_mine.wait(lock);
cv_mine.wait(lock);
waiting_mine--; crossing_mine++; consecutive_mine++;
consecutiveOpp = 0;

// writer-preference readers/writers
while (writers > 0 || waiting_writers > 0) cv.wait(lock);
readers++;
while (readers > 0 || writers > 0) cv.wait(lock);
writers = 1;
```

These are predicate questions: identify the exact Boolean that makes entry safe/fair.

2022 Merge: Fair Category Rotation

```
class Merge {
    mutex m;
    vector<int> waiting;
    vector<condition_variable> safe;
    int total=0, prev=0;
    bool merging=false;
public:
    void wait(int lane) {
        unique_lock<mutex> lock(m);
        if (!merging) { merging=true; return; }
        waiting[lane]++; total++;
        safe[lane].wait(lock); // selected lane only
    }

    void merged() {
        unique_lock<mutex> lock(m);
        if (total == 0) { merging=false; return; }
        do { prev = (prev + 1) % waiting.size(); }
        while (waiting[prev] == 0);
        waiting[prev]--; total--;
        safe[prev].notify_one(); // exactly one car
    }
};
```

Pattern: categories + fairness. Keep merging=true when handing directly to next car.

2023 Flight: Ordered Waiters

```
class Flight {
    mutex m;
    map<size_t, condition_variable> ready;
public:
    void wait_to_board(size_t pos) {
        unique_lock<mutex> lock(m);
        condition_variable mine;
        ready.emplace(pos, &mine);
        mine.wait(lock); // board_next selects me
    }

    bool board_next() {
        unique_lock<mutex> lock(m);
        if (ready.empty()) return false;
        auto it = ready.begin(); // lowest waiting position
        it->second->notify_one();
        ready.erase(it);
        return true;
    }
};
```

map sorted order solves "lowest present position first; skipped late passenger can still beat higher waiting positions."

Exact Selected Waiters: H2O Shape

```
struct Waiter {
    bool assigned = false;
    std::condition_variable cv;
};

std::mutex m;
std::deque<Waiter> H, O;

void try_group() {
    if (H.size() == 2 && O.size() >= 1) {
        Waiter *h1 = H.front(); H.pop_front();
        Waiter *h2 = H.front(); H.pop_front();
        Waiter *o = O.front(); O.pop_front();
        h1->assigned = true;
        h2->assigned = true;
        o->assigned = true;
        h1->cv.notify_one();
        h2->cv.notify_one();
        o->cv.notify_one();
    }
}

void hydrogen() {
    Waiter self;
    std::unique_lock<std::mutex> lock(m);
    H.push_back(&self);
    try_group();
    while (!self.assigned) self.cv.wait(lock);
    lock.unlock();
    bond();
}
```

- h1->cv means (*h1).cv.
- Use identity records when prompt says "exactly selected threads."

Party / Dorm Matching

```
struct Person {
    string name, result;
    bool matched=false;
    condition_variable cv;
};

mutex m;
map<Key, deque<Person>> waiting;

string meet(string name, Key mine, Key want) {
    unique_lock<mutex> lock(m);
    Key opposite = {want, mine};
    auto &q = waiting[opposite];
    if (q.empty()) {
        Person other = q.front(); q.pop_front();
        other->result = name;
        other->matched = true;
        other->cv.notify_one();
        return other->name;
    }

    Person self; self.name = name;
    waiting[mine, want].push_back(&self);
    while (!self.matched) self.cv.wait(lock);
    return self.result;
}
```

Dorm variant: key is dorm id and opposite bucket is same bucket. Use queue for FIFO within bucket.

All-Or-Nothing Inventory

```
std::mutex m;
std::condition_variable changed;
std::map<int, int> stock;

bool can_fill(vector<int> ids, vector<int> counts) {
    for (int i = 0; i < ids.size(); i++) {
        if (stock[ids[i]] < counts[i]) return false;
    }
    return true;
}

void order(vector<int> ids, vector<int> counts) {
    std::unique_lock<std::mutex> lock(m);
    while (!can_fill(ids, counts)) {
        changed.wait(lock);
    }
    // subtract only after whole order is possible
    for (int i = 0; i < ids.size(); i++) {
        stock[ids[i]] -= counts[i];
    }
}

void add(int id, int n) {
    std::unique_lock<std::mutex> lock(m);
    stock[id] += n;
    changed.notify_all(); // different orders need different items
}
```

Wrong: consume item A, then sleep waiting for item B. That creates partial reservation bugs.

Predicted Monitor Twists

```
// priority boarding with cancellation
map<int, deque<Waiter>> by_priority;
cancel(): mark waiter.cancelled=true; notify its cv
board(): skip cancelled records; notify first noncancelled

// N-of-M group formation
queue<Waiter> type[M];
if enough required types: pop exact records, assigned=true

// resource transfer A->B
while (balance[A] < amount) cv.wait(lock);
balance[A] -= amount; balance[B] += amount;
cv.notify_all();
```

Predicted pattern: if the prompt says priority/cancel/exact group/multiple resources, use waiter records plus predicates, not raw counters alone.

Last-Section Wrong-Answer Traps

```
CV wait: while (!predicate) cv.wait(lock); // not if
Inventory: check all resources, then subtract all
Flight: map.begin() gives lowest waiting key
Merge: notify one selected lane; do not notify all cars
Party/Dorm: pop waiter before notify; local waiter is safe while blocked
CalTrain: count boarding passengers, not just available seats
Deadlock: every thread must acquire locks in same global order

notify_one: selected waiter/category. notify_all: many possible predicates, each re-checks.
```

Past Final: Two-Account Transfer

```
void transfer(account *a, account *b, double amt) {
    if (a == b) return;
    account *first = a < b ? a : b;
    account *second = a < b ? b : a;
    first->mutex.lock();
    second->mutex.lock(); // global order avoids deadlock

    if (a->balance > b->balance && a->balance >= amt) {
        a->balance -= amt; b->balance += amt;
    } else if (b->balance > a->balance && b->balance >= amt) {
        b->balance -= amt; a->balance += amt;
    }
    second->mutex.unlock(); first->mutex.unlock();
}
```

Hold both locks across check and transfer; lock in address order for all callers.

Past Final: Atomic Exchange Trap

```
// x=3, y=4; two threads run:
y.exchange(x.exchange(y));

T1 inner: x=4, y=4, returns 3
T2 inner: x=4, y=4, returns 4
T1 outer: y=3
T2 outer: y=4
// final: x=4, y=4 possible. Whole expression not atomic.
```

Atomic sub-operations do not make the whole multi-step expression atomic.

V6 Constants / Struct Reminders

```
#define DISKIMG_SECTOR_SIZE 512
#define INODE_START_SECTOR 2
#define ROOT_UNNUMBER 1
#define MAX_COMPONENT_LENGTH 14

// direntv6: 2-byte inode + 14-byte name
// 512 / 16 = 32 dirents per block
// uint16_t block ptrs: 512 / 2 = 256 ptrs
```

- **TALLOC**: inode allocated.
- **IFMT**: file type mask. **IFDIR**: directory.
- **ILARG**: large V6 inode addressing.
- **size = (i_size0 <= 16) | i_size1**.

Inode *i*get: Inumber To Inode

```
int inode_iget(fs, int inumber, struct inode *inp) {
    struct Inode inodes[DISKIMG_SECTOR_SIZE /
        sizeof(struct inode)];

    int idx = inumber - 1; // inumber 1 is array index 0
    int per = DISKIMG_SECTOR_SIZE / sizeof(struct inode);
    int sector = INODE_START_SECTOR + idx / per;
    int offset = idx % per;

    if (diskimg_readsector(fs->dfd, sector, inodes)
        != DISKIMG_SECTOR_SIZE) {
        return -1;
    }
    *inp = inodes[offset];
    return 0;
}
```

Disk layout: sector 0 boot, sector 1 superblock, sector 2 starts inode list.

V6 *i*node_indexlookup

```
int inode_indexlookup(fs, struct inode *in, int b) {
    uint16_t ptrs[256], ptrs2[256];

    if (!((in->i_mode & ILARG)) {
        return in->i_addr[b]; // small: direct sectors
    }

    if (b < 7 * 256) {
        int which = b / 256; // which i_addr[0..6]
        int off = b % 256; // which entry inside it
        diskimg_readsector(fs->dfd, in->i_addr[which], ptrs);
        return ptrs[off];
    }

    b -= 7 * 256;
    int outer = b / 256;
    int inner = b % 256;

    diskimg_readsector(fs->dfd, in->i_addr[7], ptrs);
    diskimg_readsector(fs->dfd, ptrs[outer], ptrs2);
    return ptrs2[inner];
}
```

V6 large inode: **i_addr[7]** is doubly indirect. Do not treat it as a data sector.

***i*file_getblock**: Logical Block To Bytes

```
int file_getblock(fs, int inum, int lbn, void *buf) {
    struct inode in;
    if (inode_iget(fs, inum, &in) < 0) return -1;

    int size = inode_getsize(&in);
    int first = lbn * DISKIMG_SECTOR_SIZE;
    if (first >= size) return 0;

    int sector = inode_indexlookup(fs, &in, lbn);
    if (sector < 0) return -1;

    if (diskimg_readsector(fs->dfd, sector, buf)
        != DISKIMG_SECTOR_SIZE) return -1;

    int remaining = size - first;
    return remaining < DISKIMG_SECTOR_SIZE
        ? remaining
        : DISKIMG_SECTOR_SIZE;
}
```

Return valid bytes. Only the final block may be short.

Directory Scan

```
int directory_findname(fs, char *name, int dirinum,
    struct direntv6 *out) {
    struct inode dir;
    inode_iget(fs, dirinum, &dir);
    if (!((dir.i_mode & TALLOC)) return -1;
    if (!((dir.i_mode & IFMT) != IFDIR) return -1;

    int size = inode_getsize(&dir);
    int nblocks = (size + 511) / 512;
    char block[512];

    for (int b = 0; b < nblocks; b++) {
        int valid = file_getblock(fs, dirinum, b, block);
        struct direntv6 *e = (struct direntv6 *) block;
        int n = valid / sizeof(struct direntv6);

        for (int i = 0; i < n; i++) {
            if (e[i].d_inumber == 0) continue;
            if (strcmp(name, e[i].d_name, 14) == 0) {
                *out = e[i];
                return 0;
            }
        }
    }
    return -1;
}
```

Use **strncmp**: a 14-char V6 name may not have **"\0"**.

Pathname Lookup

```
int pathname_lookup(fs, char *path) {
    if (path[0] != '/') return -1;
    int cur = ROOT_UNNUMBER;

    char copy[strlen(path)+1];
    strcpy(copy, path);
    char *part = strtok(copy, "/");

    while (part != NULL) {
        struct direntv6 de;
        if (directory_findname(fs, part, cur, &de) < 0) {
            return -1;
        }
        cur = de.d_inumber;
        part = strtok(NULL, "/");
    }
    return cur;
}
```

/usr/bin/ls: root -> lookup **usr** -> lookup **bin** -> lookup **ls**.

Byte Offset To File Block

```
int off = 9999;
int lbn = off / DISKIMG_SECTOR_SIZE;
int within = off % DISKIMG_SECTOR_SIZE;

char block[512];
int n = file_getblock(fs, inum, lbn, block);
if (within < n) {
    unsigned char byte = block[within];
}
```

File offsets are logical. Inode translation turns logical block number into disk sector.

Scan All Allocated Inodes

```
int total = fs->superblock.s_isize *
    (DISKIMG_SECTOR_SIZE / sizeof(struct inode));

for (int inum = 1; inum <= total; inum++) {
    struct inode in;
    if (inode_iget(fs, inum, &in) < 0) continue;
    if (!((in.i_mode & TALLOC)) continue;

    // check property exam asks about
}
```

Use for "find file/inode/block with property X."

Does This Inode Use Disk Block X?

```
bool uses_block(fs, struct inode *in, int target) {
    int size = inode_getsize(in);
    int nblocks = (size + 511) / 512;
    for (int lbn = 0; lbn < nblocks; lbn++) {
        int sector = inode_indexlookup(fs, in, lbn);
        if (sector == target) return true;
    }
    return false;
}
```

This checks data blocks. A separate scan is needed if the question asks about indirect metadata blocks.

Find Whether Block Is Indirect

```
for each allocated inode in:
    if (!((in.i_mode & ILARG)) continue;

    // Directly named indirect blocks:
    for (int i = 0; i < 7; i++) {
        if (in.i_addr[i] == block_num) return true;
    }

    // Indirect blocks named inside doubly-indirect block:
    uint16_t ptrs[256];
    diskimg_readsector(fs->dfd, in.i_addr[7], ptrs);
    for (int i = 0; i < 256; i++)
        if (ptrs[i] == block_num) return true;
}
```

i_addr[7] itself is the doubly-indirect block; entries inside it point at singly indirect blocks.

Hard Link Skeleton

```
int target = pathname_lookup(fs, oldpath);
inode_iget(fs, target, &tin);
if ((tin.i_mode & IFMT) == IFDIR) reject;

// split new path into parent directory + final name
int parent = pathname_lookup(fs, parent_dir(newpath));
inode_iget(fs, parent, &pin);
verify parent is directory;

// scan parent directory for empty dirent slot
slot.d_inumber = target;
copy final component into slot.d_name;
tin.i_nlink++;
```

Hard link = new directory entry to same inode. No data copy, no new file inode.

Predicted V6 *i*nlink / Free Pattern

```
// unlink("/a/b/"): remove name, maybe free inode/data
parent = pathname_lookup("/a/");
scan parent directory for name "b";
victim_inum = direct_d_inumber;
direct_d_inumber = 0; // name removed
inode_iget(victim_inum, &in);
in.i_nlink--;
if (in.i_nlink == 0) {
    for each logical block lbn:
        sector = inode_indexlookup(&in, lbn);
        free_data_block(sector);
        free_indirect_metadata_blocks too;
        clear TALLOC in inode;
}
```

Crash trap: clearing directory entry before freeing blocks avoids name pointing to freed/reused data.

Add Directory Entry Safely

```
for each directory block:
    read block
    for each dirent e:
        if e.d_inumber == 0:
            e.d_inumber = target_inum;
            memset(e.d_name, 0, 14);
            strncpy(e.d_name, name, 14);
            write block back;
            return 0;

// no free slot: allocate/append a new directory block
```

Directory maps name -> inode number. Inode stores metadata and block pointers, not the filename.

Upgrade Small Inode To Large

```
uint16_t ptrs[256];

for (int i = 0; i < 8; i++) {
    ptrs[i] = in->i_addr[i]; // old direct data sectors
}
for (int i = 8; i < 256; i++) {
    ptrs[i] = 0;
}

for (int i = 0; i < 8; i++) in->i_addr[i] = 0;
in->i_addr[0] = new_indirect_sector;
in->i_mode |= ILARG;
```

Data blocks stay where they are; only pointer structure changes.

How Many Blocks Can It Address?

```
// lecture-style example: 4KB block, 4B block pointers
ptrs_per_indirect = 4096 / 4 = 1024
direct_capacity = 12
singly_capacity = 1024
doubly_capacity = 1024 * 1024

max_blocks = 12 + 1024 + 1024*1024
```

A doubly indirect block points to indirect blocks; those indirect blocks point to data blocks.

Predicted: Free-Space Bitmap/List

```
// bitmap version
for (int b = first_data; b < nblocks; b++) {
    if (bitmap_test(free, b)) {
        bitmap_clear(free, b); // now allocated
        return b;
    }
}

// linked free list version
b = super.free_head;
read block b to get next pointer;
super.free_head = next;
return b;
```

Bitmap: compact and fast to test ranges. Linked list: simple allocation, worse locality/range search.

Predicted: fsck Reachability Repair

```
clear seen_blocks;
for each allocated inode:
    for each data/indirect block reachable from inode:
        if seen[block]: report duplicate allocation;
        seen[block] = true;

for each block:
    if seen[block] && free_map says free: mark allocated
    if !seen[block] && free_map says allocated: leak -> free it
```

fsck reconstructs consistency from reachable metadata; logging instead replays committed intent.

Predicted: FAT / Linked File Traversal

```
// FAT maps block -> next block, EOF sentinel ends file
int b = inode.first_block;
while (b != EOF_BLOCK) {
    read_block(b, buf);
    consume(buf);
    b = FAT[b];
}
```

```
// random access to logical block k is O(k)
for (i = 0; i < k; i++) b = FAT[b];
```

FAT/linked files avoid inode pointer limits but random access is slower than indexed blocks.

Block Math Quick Patterns

```
blocks_needed = ceil(file_size / block_size)
internal_frag = blocks_needed * block_size - file_size
dirents_per_block = block_size / sizeof(dirent)
ptrs_per_indirect = block_size / sizeof(blockno)
max_file = (direct + ptrs + ptrs * ptrs) * block_size
```

Most filesystem arithmetic is one of these five formulas.

Past Final: Reverse Pathname

```
cur_path = "";
file = target_inum; parent = parent_inum;
while (file != ROOT_UNNUMBER) {
    scan parent directory for dirent with d_inumber == file;
    prepend "/" + dirent.d_name to cur_path;
    scan parent directory for ".." to get grandparent inum;
    file = parent;
    parent = grandparent;
}
```

Forward lookup follows names to inodes; reverse path reconstruction scans parent dirs for names pointing to known numbers.

Past Final: Erase First N Bytes

```
// file has many 512B blocks, 0 < N <= 512
if (N == 512) {
    // logical block 1 becomes logical block 0, etc.
    shift block pointers left;
    modify each indirect block + inode size;
} else {
    // byte contents cross block boundaries
    rewrite every data block after shifting bytes;
    update inode size;
}
```

Exact block erase can be pointer surgery; partial-block erase forces data copying.

Base And Bound

```
uintptr_t translate(uintptr_t va) {
    if (va >= bound) {
        trap1();
    }
    return base + va;
}
```

Message passing still works because sender traps into kernel; kernel validates/copies bytes between address spaces.

Single-Level Page Translation

```
uint64_t translate(uint64_t va, pte_t spt) {
    uint64_t off_mask = (1ULL << PAGE_BITS) - 1;
    uint64_t offset = va & off_mask;
    uint64_t vpn = va >> PAGE_BITS;

    pte_t pte = pt[vpn];
    if (!pte.valid) page_fault();
    if (!allows(pte.perm, access)) protection_fault();

    return (pte.ppn << PAGE_BITS) | offset;
}
```

- Page size 2^kk -> offset is low k bits.
- Offset is unchanged in physical address.

Multilevel Index Extraction

```
// 48-bit VA, 4KB page, 8B entries
// offset = 12 bits
// entries/table = 4096/8 = 512 = 29
// VPN bits = 48-12 = 36 -> 4 levels

offset = va & 0xfff;
L1 = (va >> 39) & 0x1fff;
L2 = (va >> 30) & 0xffff;
L3 = (va >> 21) & 0xffff;
L4 = (va >> 12) & 0xffff;
```

0x1fff is 9 one-bits. Each level indexes one page-map page.

Page Fault Handler Shape

```
handle_page_fault(va, access) {
    if (!legal_address(va, access)) {
        kill_process(); // segfault/protection fault
    }

    frame = find_free_frame();
    if (frame == NONE) {
        victim = choose_victim();
        if (victim.dirty) write_back(victim);
        invalidate_old_pte(victim);
        frame = victim.frame;
    }

    read_page_from_disk(va_page(va), frame);
    update_pte(va_page(va), frame, valid=true);
    restart_faulting_instruction();
}
```

Dirty victim must be written back. Clean victim can be discarded.

FIFO / LRU / Clock Trace Rules

```
FIFO:
    evict oldest page loaded into memory
    hits do not change queue order

LRU:
    evict least recently used page
    hits update recency

Clock:
    if ref bit == 1: clear it, advance hand
    if ref bit == 0: evict it
```

Common bug: treating FIFO hit like LRU hit. FIFO does not refresh on hit.

Worked Replacement Trace Template

```
refs: A B C A D B ...
frames: [
for each ref: -, policy = LRU/FIFO/Clock
    if present: hit; update only LRU/ref-bit
    else if free frame: fault; install
    else: fault; choose victim by policy
record after every access:
ref | hit/fault | victim | frames | metadata
```

Predicted twist: compare local vs global replacement. Local chooses victim from same process; global can evict another process.

Clock Algorithm Skeleton

```
int choose_victim() {
    while (true) {
        if (frames[hand].ref == 0) {
            int victim = hand;
            hand = (hand + 1) % nframes;
            return victim;
        }
        frames[hand].ref = 0;
        hand = (hand + 1) % nframes;
    }
}
```

On memory access, set that page frame's reference bit to 1. Clock gives recently used pages a second chance.

SRPT / Priority Trace Skeleton

```
while (unfinished_jobs exist) {
    add newly arrived jobs to ready set;
    choose ready job with smallest remaining time;
    run until next arrival or job completion;
    subtract elapsed time from remaining time;
}
```

SRPT is preemptive: a new shorter job can interrupt the currently running job.

Crash Points: Create File

Possible writes:

- 1 allocate inode / free-inode map
- 2 initialize inode
- 3 allocate data block / free-block map
- 4 write data block
- 5 update inode size/pointer
- 6 add directory entry

- Dir entry before initialized inode -> name points to garbage.
- Inode points to block marked free -> future double allocation.
- Allocated but unreachable block -> leak; usually less dangerous.

Ordered Writes: Safer Create

```
write initialized data block
write initialized inode
mark inode/block allocated
write directory entry last
```

Make reachable names point only at initialized metadata/data. For deletion, remove name before freeing inode/blocks.

Crash Recovery Answer Template

For each possible crash point:

1. List what reached disk.
2. Say whether metadata is consistent.
3. Say what is leaked, dangling, stale, or lost.
4. Say whether fsck/log replay can repair it.

Precise words score: leak = allocated but unreachable; dangling = reachable pointer to invalid/free object.

Predicted Crash Variants

```
rename(old,new):
    add new dirent -> remove old dirent
    crash between them: two names, same inode
    remove old first: possible lost file

unlink(name):
    remove dirent -> decrement link/free blocks
    crash after dirent removal: leak possible, no dangling name

append(file):
    write data block -> update inode pointer/size
    inode first can expose uninitialized/stale block
```

Exam answer: say what reached disk, what invariant broke, and whether repair needs fsck or log replay.

Redo Logging Skeleton

```
write_log("begin TM");
write_log("new inode block");
write_log("new directory block");
write_log("new free bitmap block");
write_log("commit TM");
flush_log();

install inode block to home location;
install directory block to home location;
install bitmap block to home location;
checkpoint_or_clear_log();
```

- No commit record -> ignore transaction.
- Commit present -> replay all updates.
- Replay must be idempotent because crash can happen during recovery.

Write Amplification Formula

erase unit utilization = U
usable fraction after GC = 1 - U
write amplification ≈ 1 / (1 - U)

U=.50 -> 2x
U=.90 -> 10x
U=.99 -> 100x

GC must copy valid pages elsewhere before erasing. High utilization means copy lots of live pages to reclaim tiny space.

Flash Out-Of-Place Write

```
write(logical_page L, data):
    Pnew = find_free_erased_physical_page()
    program(Pnew, data) // can change 1 bits to 0 bits
    old = map[L]
    map[L] = Pnew
    mark_garbage(old)

garbage_collect(unit):
    copy still-valid pages elsewhere
    erase entire unit // resets bits back to 1
    add pages to free pool
```

Erased flash is all 1s, not all 0s. Programming clears selected bits from 1 to 0.

Trap-And-Simulate

guest executes privileged instruction, e.g. CLI
CPU traps into hypervisor
hypervisor inspects instruction
hypervisor updates virtual CPU state
guest resumes after instruction

Guest thinks interrupts are disabled; hypervisor only disables virtual interrupts for that virtual CPU, not real machine interrupts.

System Call Trap Shape

user code:
write(fd, buf, n)

CPU:
trap into kernel mode

kernel:
validate fd and user pointer
copy bytes from user memory
ask device/filesystem to write
return result to user mode

A trap is a controlled switch from less-privileged code to privileged OS/hypervisor code.

Final Code Checklist

- execvp arrays end with NULL; child exits on failure.
- Every process closes unused pipe ends; parent closes too.
- Every child is waited for or intentionally orphaned.
- Every CV wait is while (!predicate), not if.
- All shared predicate variables use one protecting mutex.
- Exact-selection problems use waiter identity records.
- V6 directory names use bounded comparison.
- V6 code distinguishes small direct inode vs large indirect inode.
- Crash answers list writes and reason after each crash point.

Predicted: TLB / Page Walk Trace

```
access VA:
    split VA -> VPN + offset
    search TLB for VPN
    if TLB hit:
        PA = PPN_from_TLB | offset
    else:
        walk page table
        if invalid: page fault
        else install TLB entry, form PA
```

TLB entry fields: VPN tag, PPN, valid, permissions,
maybe ASID/process id to avoid flush on context switch.

Offset never changes. Permission fault is different from not-present page fault.

Predicted: Flash GC Worked Shape

erase unit has 100 pages, utilization U=.90
valid pages to copy before erase = 90
free pages gained after erase = 10
logical writes supported per GC ≈ 10
physical writes ≈ 100
write amplification ≈ 100/10 = 10x

High utilization is bad because GC copies many live pages to reclaim few free pages.

Predicted: BSD Scheduler Trace

```
for each clock tick:
    cur.recent_cpu++;

periodically:
    recent_cpu decays for all threads
    priority = base + nice + f(recent_cpu)

choose runnable thread with best effective priority
CPU-bound thread -> recent_cpu high -> worse priority
sleeping/I/O thread -> recent_cpu decays -> better priority
```

Exam wording: reducing recent CPU by blocking/sleeping can improve effective priority. nice alone is not the mechanism.

Predicted: Linking / Relocation

object file contains:
code/data bytes
symbol table: names defined/needed
relocation entries: places to patch addresses

linker:
lays out sections
resolves each symbol to an address
patches relocation/import-table entries

dynamic linking:
loader maps shared library
resolves lazy/eager imports at runtime

Relocation/import table entry = recorded spot where loader/linker must fill in the final address.

Predicted: Heap Free-List Allocator

```
malloc(n):
    need = round_up(n + header)
    find first free block with size >= need
    if large enough: split block
    mark chosen block allocated
    return pointer after header

free(p):
    h = header_before(p)
    mark h free
    if next block free: coalesce with next
    if previous block free: coalesce with previous
```

Allocator bugs: forgetting header size, not aligning, not coalescing, or leaving adjacent free blocks separate.

Predicted: Thrashing / Working Set

if working_set_pages > available_frames:
every loop iteration evicts pages needed soon
page fault rate rises sharply
CPU waits on disk instead of doing useful work

fixes:
give process more frames
reduce multiprogramming
use local replacement / working-set policy

Thrashing answer must mention repeated faults because memory cannot hold the active working set.

Past Final: Privilege / Linker Classify

privileged:
return from kernel to user, set page-map base/PML4,
disable interrupts
not privileged:
syscall/trap instruction, fill dynamic-link jump table,
invalidate own TLB entry

linker does:
combine sections, create symbol table,
patch relocation addresses, choose segment addresses
dynamic loader:
fills lazy import/jump table entries

Classification questions reward precise blast-radius reasoning: can user code harm only itself or the whole machine?

Past Final: PML Entry Address

physical address of a page-map entry:
frame number = previous-level PTE's physical frame
offset = index_within_this_level * sizeof(PTE)

for 512 entries/page and 8B PTE:
offset = 9-bit index shifted left 3
low 3 offset bits are zero

The entry itself lives in physical memory, so its address is formed like any other physical address.